

SecureLink

A self-hosted WireGuard VPN for Windows with a leak-proof kill switch
Daniyar (Danny) Abeuov · CST-452 Computer Science Capstone II · Grand Canyon University

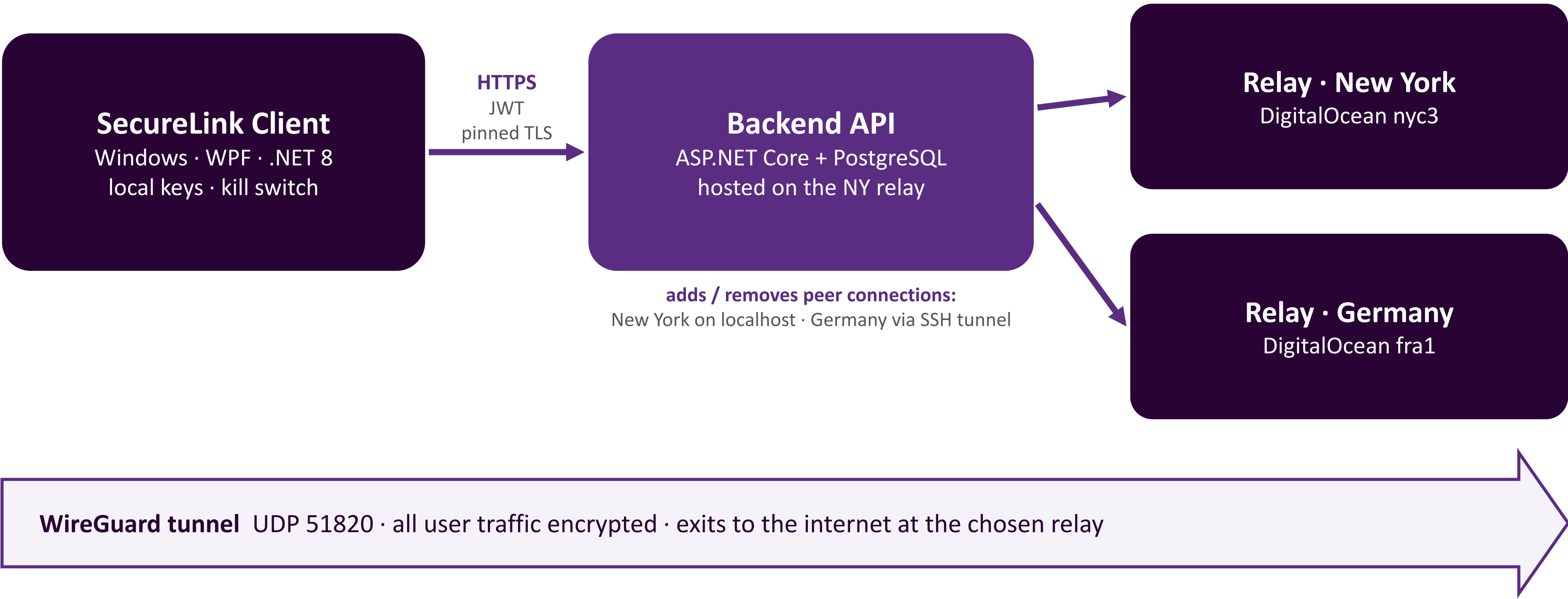


Private internet access I own end-to-end: my own encrypted WireGuard relays in New York and Frankfurt, a Windows client that sets everything up in one click, and a kill switch that never lets traffic leak.

1 Project Description

- SecureLink is a complete VPN system — desktop client, backend API and relay servers — that I designed, built and deployed.
- A Windows (WPF) app opens an encrypted WireGuard tunnel to a relay in New York or Germany.
 - Encryption keys are created on the user's PC; the private key never leaves it.
 - An ASP.NET Core backend manages accounts, enforces one session per user.
 - A kill switch blocks all traffic outside the tunnel, and the app cleans up automatically after a crash.

System Architecture



2 Project Justification

- Commercial VPNs ask you to trust a company with all of your traffic. Running your own WireGuard server saves money and ensures your data stays private.
- Control: my own servers, no third-party logging.
 - Automation: key exchange, peer setup and IP assignment happen for the user.
 - Safety: the kill switch and crash recovery prevent silent leaks.
 - Right-sized: built for a small group family and friends on two low-cost cloud servers.

3 Functional Requirements, Specifications & Standards

Functional requirements

- Register / log in (BCrypt, signed JWT)
- One active session per account
- Choose relay: New York or Germany
- Unique tunnel IP for every session
- Backend adds / removes relay peers
- Keys generated on the client only
- Connect, disconnect, live usage stats
- Kill switch blocks non-tunnel traffic
- Automatic cleanup after a crash
- Relays built by Terraform + cloud-init

Product specifications

Client: Windows 10/11 · .NET 8 WPF · WireGuard for Windows
Backend: ASP.NET Core 8 · EF Core · PostgreSQL 14 · Ubuntu 22.04
Relays: DigitalOcean nyc3 + fra1 · UDP 51820 · a /24 subnet each (253 clients)

Standards

- WireGuard
- TLS 1.2 with SHA-256
- JWT; BCrypt hashing
- Infrastructure as Code (Terraform, cloud-init)

4 Design Process

- **Plan** Proposal, user stories and functional requirements
- **Design** Architecture, data dictionary, API and security design
- **Build** Agile sprints: 25 user stories with a traceability matrix
- **Test** 37 test cases on the live system
- **Release** Milestone 6: token storage, tunnel health monitoring

5 Data Summaries

